# MLOps Platform — Operational Guide

This guide covers **two distinct operational paths** for running the platform: a fully self-contained **Synthetic Demo** for learning and evaluation, and a **Real Data Production** path for deploying with actual transaction data.

---

## Prerequisites (Both Paths)

### 1. System Requirements

| Requirement | Minimum |
|---|---|
| Python | 3.10+ |
| RAM | 4 GB |
| Disk | 2 GB free |
| OS | Windows 10+, macOS 12+, Ubuntu 20.04+ |

### 2. Install Dependencies

```
# Clone the repo
cd d:\AI STUFF\PROJECTS\MLOPS

# Create a virtual environment (recommended)
python -m venv .venv

# Activate it
# Windows:
.venv\Scripts\activate
# macOS/Linux:
# source .venv/bin/activate

# Install all dependencies
pip install -r requirements.txt
```

### 3. Environment Configuration

```
# Copy the template
copy .env.example .env     # Windows
# cp .env.example .env     # macOS/Linux
```

Edit `.env` only if you need Slack or email alerts. The defaults work for local demo usage.

> [!TIP] You do **not** need to configure Slack, email, or MLflow URIs for the synthetic demo. All defaults work out of the box.

---

## Path A: Synthetic Data Demo (Zero Setup)

This path uses auto-generated synthetic fraud data, requires **no external downloads**, and demonstrates the full end-to-end platform lifecycle in under 5 minutes.

### Step 1 — Run the Demo

```
python scripts/run_demo.py
```

**What happens automatically:**

1. A synthetic credit card fraud dataset (10,000 samples, 98% legitimate / 2% fraud) is generated
2. The baseline RandomForest model is trained and logged to MLflow
3. Reference feature distributions are saved to `data/reference/`
4. The SQLite database is initialized and pre-populated with 150 historical predictions
5. Three services start:
   - **FastAPI Serving Layer** → `http://localhost:8000`
   - **Dash Monitoring Dashboard** → `http://localhost:8050`
   - **Production Traffic Simulator** → Sends ~1 prediction/sec

### Step 2 — Explore the Dashboard

Open your browser to `http://localhost:8050`. You will see:

| Tab | What You'll See |
|---|---|
| **Overview** | Live prediction volume, fraud rate, model accuracy, average latency |
| **Data Drift** | PSI and KL divergence per feature, all green (no drift yet) |
| **Prediction Drift** | Hellinger distance, class distribution comparison |
| **Performance** | F1, precision, recall, accuracy trends |
| **Alerts** | Empty alert history |
| **Model Registry** | Version 1 marked as production |

## Step 3 — Inject Drift

When the demo console prompts you, press **ENTER** to inject severe drift. This triggers:

- **Feature shift** (magnitude 2.8) on features V1–V5
- **Label flipping** (25% of ground truth labels randomly inverted)

You can also inject drift manually in a separate terminal:

```
# Feature shift only
python scripts/inject_drift.py --type feature_shift --magnitude 2.0

# Scale change
python scripts/inject_drift.py --type scale_change --magnitude 3.0

# Gaussian noise
python scripts/inject_drift.py --type noise --magnitude 0.8

# Label flip (concept drift)
python scripts/inject_drift.py --type label_flip --ratio 0.20

# Severe (feature shift + label flip combined)
python scripts/inject_drift.py --type severe --magnitude 2.5 --ratio 0.25

# Clear all drift
python scripts/inject_drift.py --clear
```

## Step 4 — Observe the Reaction

After ~10 seconds of drifted traffic:

1. **Dashboard Data Drift tab** → PSI values spike above thresholds, features turn red
2. **Dashboard Alerts tab** → WARNING and CRITICAL alerts appear
3. **Console logs** → Retraining Decision Engine evaluates urgency
4. If urgency is HIGH → Automatic retraining pipeline triggers:
    - New model is trained on combined baseline + recent data
    - Champion/Challenger evaluation runs
    - If new model is better → promoted to production, hot-swapped into the serving layer
5. **Dashboard Model Registry tab** → Version 2 appears

## Step 5 — Manual Drift Check (API)

You can trigger an on-demand drift check via the API:

```
# Using curl
curl -X POST http://localhost:8000/drift/run
```

```
# Using Python
python -c "import requests; print(requests.post('http://localhost:8000/drift/run').json())"
```

### Step 6 — Shutdown

Press **ENTER** at the demo console prompt (or Ctrl+C). All subprocesses (API, Dashboard, Simulator) are terminated cleanly.

---

## Path B: Real Data Production Setup

This path uses the actual Kaggle Credit Card Fraud Detection dataset and is designed for realistic production-like operation.

### Step 1 — Download the Dataset

1. Go to Kaggle — Credit Card Fraud Detection
2. Download `creditcard.csv` (143 MB, 284,807 transactions)
3. Place it at:

```
d:\AI STUFF\PROJECTS\MLOPS\data\raw\creditcard.csv
```

> [!IMPORTANT] The file must be named exactly `creditcard.csv` and placed in `data/raw/`. The DataLoader automatically detects and uses this file if present, falling back to synthetic data if absent.

### Step 2 — Train the Baseline Model

```
python scripts/train_baseline.py
```

**What happens:**

1. Loads all 284,807 real transactions from `creditcard.csv`
2. Preprocesses: StandardScaler on the `Amount` column, stratified 80/20 train/test split
3. Trains a RandomForest (200 trees, depth 15, balanced class weights)
4. Logs to MLflow: params, metrics, confusion matrix plot, feature importance plot
5. Registers model as version 1, promotes to production alias
6. Saves reference distributions to `data/reference/`
7. Records model metadata in SQLite

**Expected real-data metrics (approximate):**

Expected real-data metrics (approximate):

| Metric | Expected Range |
|---|---|
| Accuracy | 99.9%+ |
| F1 (fraud) | 0.85–0.92 |
| Precision (fraud) | 0.90–0.97 |
| Recall (fraud) | 0.75–0.85 |
| AUC-ROC | 0.97–0.99 |

## Step 3 — Start the Serving Layer

```
python -m uvicorn src.api.app:create_app --host 127.0.0.1 --port 8000 --factory
```

> [!NOTE] The `--factory` flag is **required** because the app uses a factory pattern (`create_app()` returns the FastAPI instance).

## Step 4 — Start the Dashboard

```
python -m src.dashboard.app
```

Dashboard will be available at `http://localhost:8050`.

## Step 5 — Start the Traffic Simulator (Optional)

```
python scripts/simulate_production.py
```

This sends real test samples to the API at ~1 req/sec, with ground-truth feedback submitted 5 seconds after each prediction.

## Step 6 — Connect Your Own Traffic

Instead of the simulator, you can call the API directly from your application:

```
import requests

# Single prediction
response = requests.post("http://localhost:8000/predict", json={
    "features": {
        "V1": -1.3598071336738,
        "V2": -0.0727811733098497,
        "V3": 2.53634673796914,
```

```
        # ... V4 through V27 ...
        "V28": -0.0210530534538215,
        "Amount": 149.62
    }
})
result = response.json()
# {"prediction_id": 1, "predicted_label": 0, "confidence": 0.97, ...}

# Batch prediction
response = requests.post("http://localhost:8000/predict/batch", json={
    "transactions": [
        {"features": {"V1": ..., "V2": ..., ..., "Amount": ...}},
        {"features": {"V1": ..., "V2": ..., ..., "Amount": ...}},
    ]
})

# Submit ground truth (for concept drift detection)
requests.post("http://localhost:8000/ground-truth/1", json={
    "true_label": 0  # 0 = legitimate, 1 = fraud
})
```

## Step 7 — Configure Production Alerts

Edit `configs/alerting_config.yaml` and `.env`:

### Slack Integration

```
# configs/alerting_config.yaml
alerting:
  slack:
    enabled: true
    channel: "#ml-alerts"
```

```
# .env
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXXXXXXXXXXXXX
```

### Email Integration

```
# configs/alerting_config.yaml
alerting:
  email:
    enabled: true
    smtp_host: "smtp.gmail.com"
    smtp_port: 587
    smtp_user: "your-email@gmail.com"
    smtp_password: ""  # Set via SMTP_PASSWORD env var
    from_address: "mlops-monitor@yourcompany.com"
```

```
    to_addresses:
      - "team-lead@yourcompany.com"
      - "data-scientist@yourcompany.com"
```

```
# .env
SMTP_PASSWORD=your-app-password
```

## Step 8 — Tune Drift Thresholds

Edit `configs/drift_thresholds.yaml` based on your data's characteristics:

```
drift_thresholds:
  data_drift:
    psi:
      warning: 0.10    # PSI > 0.10 = moderate instability
      critical: 0.25   # PSI > 0.25 = significant shift
    kl_divergence:
      warning: 0.10
      critical: 0.20

  prediction_drift:
    hellinger:
      warning: 0.10
      critical: 0.20

  concept_drift:
    accuracy_drop:
      warning: 0.02    # 2% accuracy drop
      critical: 0.05   # 5% accuracy drop
    f1_drop:
      warning: 0.03
      critical: 0.07
```

> [!WARNING] For production systems with highly imbalanced data (like fraud), **F1 drop** is a more meaningful trigger than accuracy drop. Consider lowering the F1 threshold to 0.02/0.05 for early detection.

## Step 9 — Change Monitoring Frequency

Edit `configs/base_config.yaml`:

```
monitoring:
  check_interval_minutes: 5   # How often the scheduler runs drift checks
  window_size: 500             # Number of recent predictions to analyse
  min_samples: 100             # Minimum samples before drift check runs
```

For production, consider:

- **High-traffic APIs**: `check_interval_minutes: 1`, `window_size: 1000`
- **Low-traffic APIs**: `check_interval_minutes: 30`, `window_size: 200`

---

## Comparison: Synthetic vs Real Data

| Aspect | Path A (Synthetic) | Path B (Real Data) |
|---|---|---|
| **Dataset** | Auto-generated 10K samples | Kaggle 284K real transactions |
| **External downloads** | None | `creditcard.csv` from Kaggle |
| **Setup time** | ~1 minute | ~5 minutes |
| **Model quality** | Good for demo | Production-grade metrics |
| **Drift injection** | Supported via CLI scripts | Supported via CLI scripts or natural drift over time |
| **Alerts** | Console-only by default | Console + Slack + Email |
| **Use case** | Learning, evaluation, presentation | Company deployment, real monitoring |

---

## API Endpoints Reference

| Method | Endpoint | Description |
|---|---|---|
| `GET` | `/health` | Health check (returns `{"status": "healthy"}`) |
| `POST` | `/predict` | Single transaction prediction |
| `POST` | `/predict/batch` | Batch prediction (list of transactions) |
| `POST` | `/ground-truth/{id}` | Submit true label for a past prediction |
| `POST` | `/drift/run` | Trigger an on-demand drift check |
| `GET` | `/drift/status` | Get current drift status summary |
| `GET` | `/models/current` | Get current production model info |
| `GET` | `/models/history` | Get all model version history |
| `GET` | `/metrics/predictions` | Get prediction statistics |

## Troubleshooting

| Problem | Solution |
|---|---|
| `ModuleNotFoundError` | Ensure venv is activated and `pip install -r requirements.txt` was run |
| Port 8000 already in use | Kill existing process or change port in `configs/base_config.yaml` and `.env` |
| Port 8050 already in use | Kill existing process or change port in `configs/base_config.yaml` and `.env` |
| `FileNotFoundError: creditcard.csv` | Place the Kaggle CSV in `data/raw/creditcard.csv` or use synthetic mode |
| MLflow errors | Delete `mlruns/` directory and re-run `scripts/train_baseline.py` |
| Database locked errors | Stop all processes, delete `data/predictions.db`, restart |
| Drift not detected | Ensure `min_samples` threshold is met (default: 100 predictions with ground truth) |
| Retraining doesn't trigger | Check that urgency meets HIGH threshold in decision engine rules |

## File Structure Quick Reference

```
MLOPS/
├── configs/                    # YAML configuration files
│   ├── base_config.yaml        # Core settings (model, API, monitoring)
│   ├── drift_thresholds.yaml   # PSI, KL, Hellinger thresholds
│   └── alerting_config.yaml    # Slack, email, console alert settings
├── data/
│   ├── raw/                    # Place creditcard.csv here
│   ├── reference/              # Saved baseline feature distributions
│   └── processed/              # Saved scaler and preprocessor
├── scripts/
│   ├── run_demo.py             # One-click full demo launcher
│   ├── setup_data.py           # Generate/validate raw data
│   ├── train_baseline.py       # Train + register baseline model
│   ├── simulate_production.py  # Continuous traffic simulator
│   └── inject_drift.py         # CLI drift injection tool
├── src/
```

```
│   ├── api/                    # FastAPI serving layer
│   ├── config/                 # Pydantic settings system
│   ├── dashboard/              # Plotly Dash monitoring UI
│   ├── data/                   # Data loader, logger, drift injector
│   ├── decision/               # Retraining decision engine
│   ├── models/                 # Trainer, evaluator, MLflow registry
│   ├── monitoring/             # Drift detectors (data, prediction, concept)
│   ├── pipeline/               # Retraining pipeline + model deployer
│   └── alerting/               # Alert manager, Slack, email notifiers
├── tests/                      # Pytest automated test suite
├── docs/                       # Documentation
├── requirements.txt
└── pyproject.toml
```